

GREEN ROOF THERMAL SIMULATION

Detailed Code Documentation - Same-Structure v2

Algorithm | Complete Formula List | FVM Coefficients | TDMA Explanation

Scope of this version

This document is based on `new_baru_revised_same_structure_v2.py` and the Chagolla-Aranda et al. (2025) green roof model. It documents the same-structure Python implementation used for C3 and the legacy CAM/C3 validation path, including the algorithm, every formula used in the code, and the TDMA/Thomas algorithm used to solve tridiagonal finite-volume systems.

Teknik Mesin - Universitas Indonesia | Skripsi Green Roof | 2026

Table of Contents

Chapter	Topic	Content
1	What this documentation covers	Corrected source version, implemented scope, what is and is not solved
2	Code architecture and file role	How same-structure v2 relates to C3 and CAM branches
3	Algorithm of the model	run_validation_case(), prepare_validation_case(), run_simulation(), block coupling
4	TDMA / Thomas Algorithm	Tridiagonal matrix form, forward sweep, back substitution, why it is used
5	All formulas used by the code	Data conversion, thermodynamics, foliage, substrate heat, moisture, slab, outputs
6	Finite-volume coefficient construction	How PDEs become aw/ap/ae/bv arrays and then TDMA inputs
7	Parameters, sensors, and validation targets	Plant/substrate/slab/geometry values and NI mapping
8	Implementation corrections and differences vs paper	Dynamic T _{in} , cover fraction, moisture boundary change, RR not solved
9	Professor-ready explanation	Short answers for likely questions
A	References and source notes	Paper equations and code functions

Chapter 1: What This Documentation Covers

This is the revised documentation for the uploaded file `new_baru_revised_same_structure_v2.py`. The purpose is to make the code explainable for a professor meeting: what is solved, how the algorithm proceeds, what formulas are used, and how the TDMA solver works.

Important correction from previous documentation

The uploaded v2 file is not purely a C3-only model. It is a same-structure CAM/C3 core model. In the current workflow, it is safest to present it as the core used for C3 validation, while the separate CAM branch can be presented as the specialized branch for CAM sensor/gsw work. The v2 file still contains CAM parameters, CAM validation windows, and CAM target logic.

Status	Item	Meaning
Implemented	Green roof CAM/C3 thermal-moisture model	Foliage temperature, substrate heat, substrate water content, slab heat, q_{s_in} , Q_{gain}
Implemented	Dynamic indoor boundary	T_{in} can be read from NI sensors T1Ka for CAM and T3Kd for C3 instead of constant 25 C
Implemented	Measured/estimated parameters	Plant optics, LAI, substrate properties, weather drivers, NI validation sensors
Implemented	Numerical solvers	Implicit foliage solve and finite-volume tridiagonal systems solved using TDMA
Not implemented in this v2 file	Reference roof RR equations	Paper Eq. (15)-(17) are referenced in the paper but not solved by the current Python file
Partially different from paper	Outer Gauss-Seidel convergence loop	Paper flowchart includes repeated block coupling until convergence; this Python version updates the coupled blocks sequentially once per timestep

1.1 Main source files

File / source	Role	Notes
<code>new_baru_revised_same_structure_v2.py</code>	Core same-structure model	Defines parameters, loaders, thermodynamic helpers, foliage/substrate/slab solvers, validation runner, and plotting. This is the file documented here.
<code>run_green_roof.py</code>	Runner / presentation script	Imports the core file as <code>gr</code> , applies parameter guesses, runs C3 main and early windows, and generates validation plots.
Chagolla-Aranda et al. (2025)	Reference paper	Provides governing equations, physical assumptions, FVM/TDMA solution methodology, and equation numbering.

Chapter 2: Code Architecture and File Role

The same-structure v2 file is organized as a single complete model file. The structure below follows the actual order of the code.

Code section	Role	Main content
Section 1	Data classes + parameter initialization	PlantParameters, SubstrateParameters, SlabParameters, GeometryParameters, NumericalParameters
Section 2	Data loaders	Weather, NI/LabVIEW, LI-COR, RIKA soil moisture, synchronization, resampling
Section 3	Thermodynamic helpers	Saturation pressure, dew point, sky temperature, vapor pressure, psychrometric constant, optional dynamic h_{in}
Section 4	Foliage model	Aerodynamic resistance, stomatal resistance, foliage energy balance
Section 5	Substrate model	Moisture-dependent substrate properties, substrate heat PDE, substrate moisture PDE, TDMA solver
Section 6	Slab model	Concrete conduction and inner heat flux q_{s_in}
Section 7	Main simulation loop	Time loop, sequential block coupling, result saving, Q_{gain} integration
Section 8	LAI calibration + validation case	Optional scalar LAI calibration and validation metrics
Section 9	Visualization	6-panel CAM vs C3 plot
Section 10	Usage block	How to run CAM/C3 from Python shell

Clean presentation framing

For the professor, frame this file as: "same-structure physical model based on Chagolla-Aranda et al. (2025), updated for UI experimental data, currently used mainly for C3 validation. CAM has a separate gsw-driven branch for specialized CAM analysis."

2.1 High-level block coupling

INPUTS: plant parameters, substrate parameters, slab properties, geometry, weather, NI T_{in} , initial θ

For each timestep t :

1. Read weather: T_a , G_{sol} , RH, u , $rain_{flux}$
2. Read dynamic indoor boundary: $T_{in}(t)$ from NI, or fallback default
3. Solve foliage energy balance $\rightarrow T_f(t+dt)$
4. Solve substrate heat equation with FVM + TDMA $\rightarrow T_g(z,t+dt)$
5. Solve substrate moisture equation with FVM + TDMA $\rightarrow \theta(z,t+dt)$, j_{eva}
6. Recompute substrate thermal conductivity from new θ
7. Solve concrete slab conduction with FVM + TDMA $\rightarrow T_s(z,t+dt)$, q_{s_in}
8. Save selected outputs every 60 seconds

POST-PROCESS:

Q_{gain} = integral of q_{s_in} over time

Chapter 3: Algorithm of the Model

3.1 Algorithm: prepare_validation_case(plant_type)

```
function prepare_validation_case(plant_type, base_dir):
    plant_type = CAM or C3
    start, end = VALIDATION_WINDOWS[plant_type]

    weather = load_weather_cache_or_excel(weatherfile, start, end)
    ni = load_multiple_NI_sensor_data(NI_files)
    ni = ni clipped to start/end

    rika = load_soil_moisture_data(datasoilmoisture.zip, plant_type)
    rika = rika clipped to start/end
    theta_initial = get_theta_initial_from_rika(rika, start)

    return weather, ni, rika, theta_initial
```

3.2 Algorithm: run_validation_case(plant_type)

```
function run_validation_case(plant_type, calibrate_lai=False):
    plant = bromelia if CAM else wedelia
    weather, ni, rika, theta_initial = prepare_validation_case(plant_type)

    if CAM:
        T_in_series = ni['T_in_CAM']
        target_col = VALIDATION_TARGETS['CAM'] # default v2: T2A2
        T_g_top_initial = first valid ni['T_g_top_CAM']
    else:
        T_in_series = ni['T_in_C3']
        target_col = VALIDATION_TARGETS['C3'] # default v2: T3Ka
        T_g_top_initial = None

    T_s_in_initial = first valid value from ni[target_col]

    if calibrate_lai:
        LAI_opt, results = calibrate_LAI(...)
    else:
        results = run_simulation(...)

    metrics = validation_metrics(model T_s_in, ni[target_col])
    add LAI_used, window_start, window_end to metrics
    return results, metrics
```

3.3 Algorithm: run_simulation()

```
function run_simulation(weather_df, plant, substrate, slab, geom, num, theta_initial, T_in_series):
    verify H_g and H_slab are defined
    weather_1s = prepare_weather_1s(weather_df, dt)
    if T_in_series exists: resample/interpolate it to dt and align with weather_1s

    initialize:
        T_a_init = first weather air temperature in Kelvin
        T_g = linear profile from T_g_top_initial to T_a_init
        T_s = linear profile from T_a_init to T_s_in_initial
        T_f = T_a_init
        theta = make_theta_initial_profile(theta_initial)

    for each timestamp ts and weather row:
```

```

T_a_K = row['T_a'] + 273.15
G_sol = max(row['G_sol'], 0)
RH = clip(row['RH'], 1, 99)
u = max(row['u'], 0.1)
j_pr = j_irrigation + max(row['rain_flux'], 0)
T_in_current = T_in_series(ts) + 273.15, else geom.T_in_default
theta_avg = mean(theta)

T_f = solve_foliage_temperature(...)
T_g = solve_substrate_heat(...)
theta, j_eva = solve_substrate_moisture(...)
lambda_g = compute_substrate_properties(theta).lambda_g
T_s, q_s_in = solve_slab_heat(...)

if step is save step:
    store datetime, temperatures, theta, q_s_in, j_eva, T_a, G_sol, T_in_used, j_pr

Q_gain = trapezoid(q_s_in over saved time)
if A_roof exists: Q_gain_total = Q_gain * A_roof
return results

```

3.4 Algorithmic note: paper vs current Python coupling

The reference paper states that the discretized systems are solved with TDMA employing a Gauss-Seidel approach and repeated until a convergence criterion of $1e-5$ is met. The uploaded Python v2 code stores a convergence parameter, but the actual `run_simulation()` loop does not implement an outer repeated block-coupling convergence loop. Instead, it performs one sequential update of foliage, substrate heat, substrate moisture, and slab heat per timestep. Each PDE block is still solved implicitly through TDMA.

Chapter 4: TDMA / Thomas Algorithm

TDMA stands for Tridiagonal Matrix Algorithm. It is also called the Thomas Algorithm. It is used because the 1D finite-volume discretization creates equations where each node only depends on the west node, itself, and the east node. That produces a tridiagonal matrix instead of a full matrix.

4.1 Tridiagonal system form

For each node i , the linearized finite-volume equation can be written as:

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$$

For the entire grid, this becomes:

$$\begin{bmatrix} b_0 & c_0 & 0 & 0 & \dots \\ a_1 & b_1 & c_1 & 0 & \dots \\ 0 & a_2 & b_2 & c_2 & \dots \\ \dots & & & & \\ \dots & 0 & a_n & b_n & \dots \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \dots \\ x_n \end{bmatrix} = \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ \dots \\ d_n \end{bmatrix}$$

4.2 TDMA algorithm used in the code

Stage	Formula / code step	Meaning
Initialize	$c_prime[0] = c[0] / b[0]$	Normalize the first upper-diagonal coefficient
Initialize	$d_prime[0] = d[0] / b[0]$	Normalize the first right-hand-side value
Forward sweep	$denom = b[i] - a[i] * c_prime[i-1]$	Eliminate lower diagonal coefficient row by row
Forward sweep	$c_prime[i] = c[i] / denom$	Update effective upper coefficient
Forward sweep	$d_prime[i] = (d[i] - a[i] * d_prime[i-1]) / denom$	Update effective RHS
Back substitution	$x[n-1] = d_prime[n-1]$	Last unknown is now directly available
Back substitution	$x[i] = d_prime[i] - c_prime[i] * x[i+1]$	Solve unknowns from bottom to top

4.3 Exact TDMA pseudocode

```
function tdma_solver(a, b, c, d):
    n = length(d)
    c_prime = zeros(n)
    d_prime = zeros(n)
    x = zeros(n)

    c_prime[0] = c[0] / b[0]
    d_prime[0] = d[0] / b[0]

    for i = 1 to n-1:
        denom = b[i] - a[i] * c_prime[i-1]
        if abs(denom) < 1e-30: denom = 1e-30
        c_prime[i] = c[i] / denom
        d_prime[i] = (d[i] - a[i] * d_prime[i-1]) / denom

    x[n-1] = d_prime[n-1]
    for i = n-2 down to 0:
        x[i] = d_prime[i] - c_prime[i] * x[i+1]

    return x
```

4.4 Why off-diagonal coefficients are negative in the code

The code builds a_w and a_e as positive conductance values. In the algebraic matrix form, neighbor terms are moved to the left-hand side as negative off-diagonal coefficients. Therefore the solver is called as `tdma_solver(-aw, ap, -ae, bv)`. This sign convention is important; otherwise heat diffusion can flow in the wrong numerical direction.

Chapter 5: All Formulas Used by the Code

Notation

The formulas below are written in code-execution order. Temperature in the solver is mainly Kelvin, while plotted output is Celsius. z is vertical depth, θ is volumetric water content, and u is wind speed. Paper equation numbers refer to Chagolla-Aranda et al. (2025).

5.1 Data conversion, resampling, and initialization

ID	Formula used	Paper / source	Role in code
D1	$T_K = T_C + 273.15$	-	Convert Celsius weather/NI data to Kelvin for radiation and vapor pressure
D2	$\text{rain_flux} = \text{rain} / 60$	-	Convert mm per minute rain to $\text{kg m}^{-2} \text{s}^{-1}$ water flux, because $1 \text{ mm water} = 1 \text{ kg m}^{-2}$
D3	$u = \max(u, 0.1)$	-	Avoid zero wind speed in aerodynamic resistance
D4	$\text{RH} = \text{clip}(\text{RH}, 1, 99)$	-	Avoid impossible RH and singular vapor behavior
D5	$j_{\text{pr}} = j_{\text{irrigation}} + \max(\text{rain_flux}, 0)$	Eq. (9)	Water input at substrate top
D6	$\text{dz_g} = H_{\text{g}} / (Nz_{\text{substrate}} - 1)$	Sec. 3.1	Substrate grid spacing
D7	$\text{dz_s} = H_{\text{slab}} / (Nz_{\text{slab}} - 1)$	Sec. 3.1	Slab grid spacing
D8	$T_{\text{g}}(z,0) = \text{linear}(T_{\text{g_top_initial}}, T_{\text{a_init}})$	-	Initial substrate temperature profile
D9	$T_{\text{s}}(z,0) = \text{linear}(T_{\text{a_init}}, T_{\text{s_in_initial}})$	-	Initial concrete slab temperature profile
D10	$\theta(z,0) = 0.8 \theta_{\text{sat}}$ if no RIKA profile	-	Fallback initial VWC profile
D11	$\theta(z,0) = \text{interp}([2\text{cm}, 7\text{cm}], [\theta_{\text{2cm}}, \theta_{\text{7cm}}])$	-	Initial VWC profile if RIKA sensor data exists

5.2 Thermodynamic helper formulas

ID	Formula used	Paper / source	Role in code
T1	$T_C = T_K - 273.15$	-	Kelvin to Celsius for empirical equations
T2	$P_{\text{sat}} = 610.78 \exp(17.269 T_C / (T_C + 237.29))$	-	Magnus saturation vapor pressure [Pa]
T3	$\gamma_{\text{dp}} = a T_C / (b + T_C) + \ln(\text{RH}/100)$	-	Dew-point helper, $a=17.27$, $b=237.7$
T4	$T_{\text{dp}} = b \gamma_{\text{dp}} / (a - \gamma_{\text{dp}})$	-	Dew point in Celsius
T5	$\epsilon_{\text{sky}} = 0.711 + 0.56(T_{\text{dp}}/100) + 0.73(T_{\text{dp}}/100)^2$	-	Effective sky emissivity from dew point
T6	$T_{\text{sky}} = \epsilon_{\text{sky}}^{(1/4)} T_{\text{a}}$	-	Effective sky temperature [K]
T7	$P_{\text{a}} = (\text{RH}/100) P_{\text{sat}}(T_{\text{a}})$	-	Ambient vapor pressure [Pa]
T8	$\text{VPD} = \max(P_{\text{sat}}(T_{\text{surface}}) - P_{\text{a}}, 0)$	-	Vapor pressure deficit [Pa]
T9	$\gamma = 1005 \cdot 101325 / (0.622 \cdot 2.45e6)$	-	Psychrometric constant, approximately 66.8 Pa/K
T10	$h_{\text{in,dyn}} = a T_{\text{surface}} - T_{\text{air}} ^{(1/3)}$	-	Optional natural-convection h_{in} , $a=0.76$ stable or 1.52 unstable, clipped $0.5\text{--}4 \text{ W/m}^2\text{K}$

5.3 Foliage model formulas

ID	Formula used	Paper / source	Role in code
F1	$d0 = 0.701 H_{\text{f}}^{0.975}$	A.3	Canopy displacement height
F2	$Z0 = 0.131 H_{\text{f}}^{0.997}$	A.4	Canopy roughness length
F3	$r_{\text{a}} = [\ln((z_{\text{ref}} - d0)/Z0)]^2 / (k^2 \max(u, 0.1))$	A.2	Aerodynamic resistance; wind speed enters here
F4	$f1 = 1 + \exp[-0.034(G_{\text{sol}} - 3.5)]$	A.7	Light response for stomatal resistance
F5	$f2 = [\exp(0.3T_C) + 258] / [\exp(0.3T_C) + 27]$	A.8	Temperature response
F6	$f3 = 4e-3 + \exp[-0.73 \cdot 0.622e3 / 101325 \cdot \text{VPD}]$	A.9	VPD response
F7	$\theta_{\text{safe}} = \max(\theta_{\text{a}}, \theta_{\text{min}} + 0.001)$	-	Avoid division by zero in moisture stress
F8	$f4 = \theta_{\text{sat}} / \theta_{\text{safe}}$	A.10 approx.	Soil moisture stress function
F9	$r_{\text{stoma}} = (r_{\text{stoma_min}}/LAI) f1 f2 f3$	A.6	Actual stomatal resistance

ID	Formula used	Paper / source	Role in code
F10	f4 if CAM and G_sol > 50: r_stoma = 12 r_stoma	-	CAM daytime stomatal closure multiplier in same-structure model
F11	if CAM and G_sol <= 50: r_stoma = 0.5 r_stoma	-	CAM nighttime stomatal opening multiplier
F12	r_stoma = max(r_stoma, 5)	-	Lower numerical bound
F13	epsilon_fg = 1/(1/epsilon_f + 1/epsilon_g - 1)	Eq. (3)	Effective foliage-substrate emissivity
F14	R_f_net = alpha_f G_sol - epsilon_f sigma(T_f^4 - T_sky^4) - epsilon_fg sigma(T_f^4 - T_g^4)	Eq. (2)	Net radiation on foliage
F15	h_conv_f = LAI (rho_air cp_air) / r_a	A.1	Foliage convection coefficient
F16	h_eva_f = (LAI/gamma)(rho_air cp_air)/(r_a + r_stoma)	A.5	Foliage evaporation coefficient
F17	dPdT = P_sat(T_f+0.5) - P_sat(T_f-0.5)	-	Numerical slope of saturation pressure
F18	A = rho_cp_f d_eff/dt + h_conv_f + h_eva_f dPdT	Eq. (1) implicit	Left coefficient for foliage implicit solve
F19	B = rho_cp_f d_eff/dt T_f_old + R_f_net + h_conv_f T_a + h_eva_f(P_a - P_f_sat + dPdT T_f_old)	Eq. (1) implicit	Right-hand side for foliage implicit solve
F20	T_f_new = B/A	Eq. (1)	Updated foliage temperature

5.4 Substrate property and surface resistance formulas

ID	Formula used	Paper / source	Role in code
S1	theta_safe = max(theta, theta_min + 1e-6)	-	Safe VWC
S2	S = clip(theta_safe/theta_sat, 0, 1)	-	Degree of saturation
S3	(rho_cp)_g = cp_g (0.2 + theta_safe) rho_g	B.8	Volumetric heat capacity of substrate
S4	k_theta = k_theta_sat S^(2b+3)	B.10	Hydraulic conductivity
S5	lambda_sat = lambda_dry + theta_sat lambda_water if not specified	-	Default saturated thermal conductivity estimate
S6	lambda_g = lambda_dry + S(lambda_sat - lambda_dry)	Code correction to B.9	Thermal conductivity bounded between dry and saturated condition
S7	D_theta = b k_theta_sat psi_sat / theta_safe * S^(b+3)	B.11 sign-adjusted	Hydraulic diffusivity using positive suction magnitude
S8	r_vap = 100 S_surface^(-2)	B.7 simplified	Moisture-dependent substrate vapor resistance

5.5 Substrate heat-transfer formulas

ID	Formula used	Paper / source	Role in code
H1	G_to_substrate = [(1-cover) + cover tau_f] G_sol	Modified Eq. (6)	Solar reaching substrate with incomplete canopy cover
H2	R_g_net = (1 - rho_g_rad)G_to_substrate + epsilon_fg sigma(T_f^4 - T_g^4)	Eq. (6) modified	Net radiation at substrate surface
H3	Z_M = 0.131 H_f^0.997; Z_u = H_f + 2	B.5 support	Momentum roughness and reference height
H4	u_f = u * ln((H_f-d0)/Z_M) / ln((Z_u-d0)/Z_M)	B.5	Wind velocity over foliage surface
H5	a_drag = sqrt(0.28 LAI H_f d_f)	B.6	Canopy drag coefficient
H6	u_g = max(u_f exp[-a_drag(1 - 0.05/H_f)], 0.01)	B.4	Wind under canopy / at substrate
H7	r_c = 1/(0.004 + 0.012 u_g)	B.3	Resistance between substrate and foliage air
H8	h_conv_g = LAI(rho_air cp_air)/(r_a + r_c)	B.1	Substrate convection coefficient
H9	h_eva_g = (LAI/gamma)(rho_air cp_air)/(r_a + r_vap)	B.2	Substrate evaporation coefficient
H10	q_top = R_g_net - h_conv_g(T_g0 - T_a) - h_eva_g max(P_g_sat - P_a, 0)	Eq. (5)	Top substrate boundary heat flux
H11	lambda_w = 0.5(lambda_i-1 + lambda_i)	FVM	West interface thermal conductivity
H12	lambda_e = 0.5(lambda_i + lambda_i+1)	FVM	East interface thermal conductivity
H13	aw_i = lambda_w/dz^2; ae_i = lambda_e/dz^2	Eq. (4)	Internal node neighbor conductances
H14	ap_i = rho_cp_g i/dt + aw_i + ae_i	Eq. (4)	Internal node main diagonal
H15	bv_i = rho_cp_g i/dt T_g_i_old	Eq. (4)	Internal node RHS
H16	top: ae_0=lambda_g0/dz^2; ap_0=rho_cp_g0/dt+ae_0; bv_0=rho_cp_g0/dt*T_g0+q_top/dz	Eq. (5)	Top boundary coefficient

ID	Formula used	Paper / source	Role in code
H17	bottom: $aw_N = \lambda_{gN}/dz^2$; $ap_N = \rho_{cp_gN}/dt + aw_N + \lambda_{gN}/dz^2$; $bv_N = \rho_{cp_gN}/dt + T_{gN} + \lambda_{gN}/dz^2 * T_{slab_top}$	Eq. (7)	Bottom boundary coupled to slab-top temperature
H18	$T_{g_new} = TDMA(-aw, ap, -ae, bv)$	TDMA	Updated substrate temperature profile

5.6 Substrate moisture formulas

ID	Formula used	Paper / source	Role in code
M1	$P_{f_sat} = P_{sat}(T_f)$; $P_{g_sat} = P_{sat}(T_{g_surface})$	-	Saturation vapor pressures for foliage and substrate
M2	$r_{stoma} = \text{compute_stomatal_resistance}(G_{sol}, T_f, P_{f_sat}, P_a, \text{mean}(\theta))$	A.6-A.10	Stomatal resistance used again in water balance
M3	$h_{eva_f} = (LA/\gamma)(\rho_{air_cp_air})/(r_a + r_{stoma})$	A.5	Foliage evapotranspiration coefficient
M4	$h_{eva_g} = (LA/\gamma)(\rho_{air_cp_air})/(r_a + r_{vap})$	B.2	Substrate evaporation coefficient
M5	$j_{eva_f} = \max(0, h_{eva_f}(P_{f_sat} - P_a)/l_{fg})$	Eq. (10)	Foliage water mass flux
M6	$j_{eva_g} = \max(0, h_{eva_g}(P_{g_sat} - P_a)/l_{fg})$	Eq. (10)	Substrate water mass flux
M7	$j_{eva} = j_{eva_f} + j_{eva_g}$	Eq. (10)	Total evapotranspiration
M8	$j_{eva_ms} = j_{eva}/\rho_{water}$; $j_{irrig_ms} = j_{irrigation}/\rho_{water}$	-	Convert $kg\ m^{-2}\ s^{-1}$ to $m\ s^{-1}$ water depth flux
M9	$Dw = 0.5(D_{i-1} + D_i)$; $De = 0.5(D_i + D_{i+1})$	Eq. (8)	Diffusivity at west/east faces
M10	$kw = 0.5(k_{i-1} + k_i)$; $ke = 0.5(k_i + k_{i+1})$	Eq. (8)	Hydraulic conductivity at faces
M11	$aw_i = Dw/dz^2$; $ae_i = De/dz^2$	Eq. (8)	Moisture diffusion neighbor coefficients
M12	$ap_i = 1/dt + aw_i + ae_i$	Eq. (8)	Moisture main diagonal
M13	$bv_i = \theta_i/dt + (ke - kw)/dz$	Eq. (8)	Moisture RHS including gravity conductivity gradient
M14	$j_{net} = j_{irrig_ms} - j_{eva_ms}$	Eq. (9)	Net top water input; positive means infiltration
M15	top: $ae_0 = D_0/dz^2$; $ap_0 = 1/dt + ae_0$; $bv_0 = \theta_0/dt + (j_{net} - k_{\theta_0})/dz$	Eq. (9) corrected	Top water balance; gravity conductivity is not artificial source
M16	bottom: $aw_N = D_N/dz^2$; $ap_N = 1/dt + aw_N$; $bv_N = \theta_N/dt - k_{\theta_N}/dz$	Eq. (11) modified	Free drainage / unit-gradient drainage
M17	$\theta_{new} = TDMA(-aw, ap, -ae, bv)$	TDMA	Updated VWC profile
M18	$\theta_{new} = \text{clip}(\theta_{new}, \theta_{min}, \theta_{sat})$	-	Physical bounds on VWC

5.7 Slab heat-transfer formulas

ID	Formula used	Paper / source	Role in code
C1	$\rho_{cp_s} = \rho_s\ cp_s$	Eq. (12)	Concrete volumetric heat capacity
C2	$aw_i = \lambda_{s_s}/dz^2$; $ae_i = \lambda_{s_s}/dz^2$	Eq. (12)	Internal slab conductances
C3	$ap_i = \rho_{cp_s}/dt + aw_i + ae_i$	Eq. (12)	Internal slab main diagonal
C4	$bv_i = \rho_{cp_s}/dt + T_{s_i_old}$	Eq. (12)	Internal slab RHS
C5	$\lambda_{w_bc} = \lambda_{g_bottom}/dz^2$	Eq. (13)	Top slab coupling to substrate bottom
C6	top: $ae_0 = \lambda_{s_s}/dz^2$; $ap_0 = \rho_{cp_s}/dt + ae_0 + \lambda_{w_bc}$; $bv_0 = \rho_{cp_s}/dt + T_{s0} + \lambda_{w_bc} * T_{g_bottom}$	Eq. (13)	Top slab boundary condition
C7	$h_{in} = \text{geom.h}_{in}$, unless $\text{dynamic.h}_{in} = \text{True}$	Eq. (14)	Indoor convection coefficient
C8	bottom: $aw_N = \lambda_{s_s}/dz^2$; $ap_N = \rho_{cp_s}/dt + aw_N + h_{in}/dz$; $bv_N = \rho_{cp_s}/dt + T_N + h_{in} * T_{in}/dz$	Eq. (14)	Bottom slab boundary condition
C9	$T_{s_new} = TDMA(-aw, ap, -ae, bv)$	TDMA	Updated slab temperature profile
C10	$q_{s_in} = h_{in}(T_{s_new}[-1] - T_{in})$	Eq. (14)	Heat flux to interior; positive = heat into interior

5.8 Post-processing, validation, and plotting formulas

ID	Formula used	Paper / source	Role in code
P1	$Q_{gain} = \text{trapezoid}(q_{s_in}, dx = \text{save_every_s})$	Eq. (18)	Net heat gain per roof area [$J\ m^{-2}$]

ID	Formula used	Paper / source	Role in code
P2	$Q_gain_total = Q_gain * A_roof$	-	Total heat gain for actual roof plan area [J]
P3	$bias = mean(T_model - T_measured)$	-	Validation bias
P4	$MAE = mean(T_model - T_measured)$	-	Mean absolute error
P5	$RMSE = sqrt(mean((T_model - T_measured)^2))$	-	Root mean square error
P6	$ET_hourly = j_eva * 3600$	-	Convert $kg\ m^{-2}\ s^{-1}$ to $kg\ m^{-2}\ h^{-1}$ for plotting
P7	$time_h = time_s / 3600$	-	Convert simulation time to hours for plotting
P8	$alpha_f = 1 - rho_f - tau_f$	-	Leaf absorptivity consistency check
P9	$r_stoma_min = percentile_5(1/(gsw*0.0224))$	-	LI-COR conversion for stomatal resistance

Chapter 6: Finite-Volume Coefficient Construction

The heat and moisture PDEs are not solved by directly differentiating arrays. They are first transformed into algebraic coefficients for each computational node. The arrays *aw*, *ap*, *ae*, and *bv* represent west coefficient, main diagonal coefficient, east coefficient, and right-hand side.

Symbol	Meaning	Code interpretation
<i>aw</i>	West/previous-depth neighbor coefficient	Connects node <i>i</i> to node <i>i-1</i>
<i>ap</i>	Main diagonal coefficient	Coefficient of current node <i>i</i>
<i>ae</i>	East/next-depth neighbor coefficient	Connects node <i>i</i> to node <i>i+1</i>
<i>bv</i>	Right-hand side vector	Known values from previous timestep and boundary fluxes
TDMA input	<code>tdma_solver(-aw, ap, -ae, bv)</code>	Because neighbor coefficients become negative off-diagonal terms in matrix form

6.1 Generic internal heat node

```

For substrate heat or slab heat at internal node i:
  aw[i] = lambda_w / dz^2
  ae[i] = lambda_e / dz^2
  ap[i] = rho_cp/dt + aw[i] + ae[i]
  bv[i] = rho_cp/dt * T_old[i]

Matrix row:
  -aw[i] T[i-1] + ap[i] T[i] - ae[i] T[i+1] = bv[i]

```

6.2 Generic internal moisture node

```

For water content theta at internal node i:
  aw[i] = D_w / dz^2
  ae[i] = D_e / dz^2
  ap[i] = 1/dt + aw[i] + ae[i]
  bv[i] = theta_old[i]/dt + (k_e - k_w)/dz

Matrix row:
  -aw[i] theta[i-1] + ap[i] theta[i] - ae[i] theta[i+1] = bv[i]

```

6.3 Boundary nodes

Boundary nodes differ from internal nodes because fluxes or known boundary temperatures appear as source terms in *bv*, and sometimes as added conductance in *ap*. Examples: q_{top}/dz enters the top substrate heat boundary; $h_{in}T_{in}/dz$ enters the bottom slab boundary; $(j_{net}-k_{theta}[0])/dz$ enters the top moisture boundary.

Chapter 7: Parameters, Sensors, and Validation Targets

7.1 Main parameter values in v2 core file

Parameter	Value	Notes
Bromelia CAM H _f	0.098 m	Plant measurement from previous data
Bromelia CAM LAI	1.95 initially	Can be overwritten by ImageJ or calibration
Bromelia CAM cover_fraction	0.95 in core; may be overwritten by runner	Projected canopy cover fraction, not LAI
Bromelia CAM r_stoma_min	400 s/m placeholder	Needs confirmed CAM LI-COR extraction
Wedelia C3 H _f	0.276 m	Plant measurement from previous data
Wedelia C3 LAI	1.3 initially	Can be overwritten by ImageJ or calibration
Wedelia C3 cover_fraction	0.5 initially; apply_scientific_guess sets 0.45	Photo/ImageJ-driven estimate
Wedelia C3 r_stoma_min	167.2 s/m	Derived from C3 LI-COR
Substrate lambda_dry	0.2 in initialization; apply_scientific_guess default 0.12	Use runtime printed value when reporting
Substrate theta_sat	0.87 in initialization; apply_scientific_guess default 0.90	Use runtime printed value
Substrate rho_g	400 kg/m ³	Scientific guess / data
Substrate k_theta_sat	5e-6 m/s	Hydraulic conductivity estimate
geom.H_g	0.06 in initialization; apply_scientific_guess default 0.10	Substrate depth - must be stated carefully
slab.H_slab	0.10 m	Concrete slab thickness
geom.h_in	8 W/m ² K default	Used unless dynamic_h_in=True

Presentation warning

Because `run_green_roof.py` can overwrite values in the core file, do not quote a parameter only from the class initialization. Quote the printed runtime values from the exact script run, especially `H_g`, `lambda_dry`, `theta_sat`, `LAI`, and `cover_fraction`.

7.2 Weather inputs

Variable	Meaning	Where used
T _a	Ambient air temperature	Used in foliage, substrate heat, vapor pressure, initialization
RH	Relative humidity	Used for vapor pressure, VPD, sky temperature
u	Wind speed	Used in aerodynamic resistance r _a and under-canopy wind u _f /u _g
rain	Rain per interval	Converted to rain_flux for moisture boundary
G _{sol}	Solar radiation	Used in foliage radiation, substrate radiation, stomatal light response

7.3 NI sensor mapping

Channel	Physical location	Code alias	Use
T1Ke	Tanah atas CAM	T_g_top_CAM	Initial/top substrate target for CAM
T2A	Tanah bawah CAM	T_g_bot_CAM	Filtered for anomalies; bottom substrate diagnostic
T1Tb	Atap indoor CAM	T_s_in_CAM	Indoor slab/roof target candidate
T1Ta	Atap outdoor CAM	T_s_ext_CAM	Outdoor surface diagnostic
T1Ka	Ruangan CAM	T_in_CAM	Dynamic indoor boundary for CAM
T3Ka	Atap indoor C3	T_s_in_C3	Default C3 validation target
T3Kd	Ruangan C3	T_in_C3	Dynamic indoor boundary for C3
T2Ka	Atap indoor RR	T_r_in_RR	Reference roof sensor only, not solved in current v2 GR code

Chapter 8: Implementation Corrections and Differences vs Paper

ID	Correction / change	Meaning
C1	Dynamic T_{in}	Paper assumes room at 25 C. Code can use NI T1Ka/T3Kd every timestep. This is important because the test boxes are outdoor and not air-conditioned.
C2	Plant-specific τ_f	C3 uses larger transmissivity than CAM due to thinner leaves. CAM retains lower transmissivity.
C3	α_f from ρ_f and τ_f	$\alpha_f = 1 - \rho_f - \tau_f$ to keep optical properties physically consistent.
C4	C3 stomatal resistance from LI-COR	r_{stoma_min} C3 = 167.2 s/m from gsw conversion; CAM still placeholder in same-structure branch.
C5	NI XML parser	Reads the xlsx internal XML directly, avoiding truncated extraction.
C6	Temperature anomaly filtering	T2A and general temperature channels outside physical range are replaced with NaN and interpolated for short gaps.
C7	Timestamp synchronization	NI and Davis are resampled/aligned to common 1-minute and then 1-second simulation grid.
C8	CAM r_{stoma} placeholder	Same-structure model still uses simplified CAM multiplier; specialized CAM branch is more defensible for CAM presentation.
V2-specific	Canopy cover fraction	Substrate solar input uses $((1-cover) + cover * \tau_f) G_{sol}$, so uncovered substrate receives direct solar.
V2-specific	Moisture top boundary correction	Top moisture RHS uses $(j_{net-k_theta[0]})/dz$, avoiding conductivity as artificial water source.
V2-specific	Free drainage bottom boundary	Bottom moisture boundary uses drainage term every timestep, instead of waiting until θ is nearly saturated.
V2-specific	Optional dynamic h_{in}	$dynamic_h_{in}$ can compute natural convection h from Delta T, but default is False and $h_{in}=8$ is used.

8.1 Equations from the paper not used in the current v2 code

Paper equation	Description	Status
Eq. (15)	Reference roof slab PDE	Not implemented in current same-structure GR simulation path
Eq. (16)	Reference roof exterior boundary	Not implemented in current v2 code
Eq. (17)	Reference roof interior boundary	Not implemented in current v2 code; only GR slab Eq. (14) is used

Chapter 9: Professor-Ready Explanation

Question	Short answer
What is the algorithm?	The code uses a sequential coupled algorithm: load weather/NI data, initialize profiles, then for every timestep solve foliage energy balance, substrate heat, substrate moisture, and slab heat. The PDE blocks are discretized with FVM and solved by TDMA.
Is wind used?	Yes. Wind speed u is used in r_a , u_f , u_g , and r_c . It affects foliage convection, foliage evapotranspiration, substrate convection, and substrate evaporation. It does not directly appear in slab conduction.
Why TDMA?	Because the 1D FVM discretization produces tridiagonal systems. TDMA solves them in $O(n)$, much faster and cleaner than full matrix inversion.
What is the key output?	T_{s_in} and q_{s_in} . $q_{s_in} = h_{in}(T_{s_in} - T_{in})$. Positive means heat enters the interior, negative means heat leaves the interior. Q_{gain} integrates q_{s_in} over time.
What is new compared to the paper?	The boundary T_{in} is dynamic from NI data, plant optical parameters are adapted to Bromelia and Wedelia, substrate is soil+rice husk, and measured validation data are used from UI test boxes.
What should be stated honestly?	This v2 file is a same-structure physical model. It is useful for C3 validation, while CAM needs careful interpretation because the CAM stomatal value is still placeholder in this branch.
Is the paper flowchart exactly reproduced?	Not exactly. The paper describes repeated Gauss-Seidel coupling until convergence; the Python v2 code performs sequential block updates per timestep, but solves each tridiagonal block implicitly using TDMA.
What formula should I remember?	$q_{s_in} = h_{in}(T_{s_in} - T_{in})$. This connects the simulated slab temperature to heat gain/loss into the box.

9.1 Suggested oral explanation

The model follows the Chagolla-Aranda green roof structure. I divide the system into three coupled blocks: foliage, substrate, and concrete slab. Weather data provide air temperature, solar radiation, RH, wind, and rain. At every timestep, the foliage energy balance is solved first, then the substrate heat equation, then the substrate moisture equation, and finally the slab conduction equation. The substrate and slab are solved using finite volume discretization, producing tridiagonal systems that are solved with TDMA. The main output is T_{s_in} and q_{s_in} , where positive q_{s_in} means heat entering the interior. My correction is that T_{in} is not assumed constant at 25 C; it is taken dynamically from NI sensors because the test box is outdoors.

Appendix A: References and Source Notes

Type	Reference	Use
Primary code	new_baru_revised_same_structure_v2.py	Uploaded source file documented here.
Main paper	Chagolla-Aranda, M.A., Sima, E., Hernandez-Lopez, I., Pina-Ortiz, A., & Avila-Hernandez, A. (2025). Dynamic thermal evaluation of a green roof system under warm weather conditions: Case of Tropical and Dry climates. Journal of Building Engineering, 103, 112053.	Source of governing equations, FVM/TDMA methodology, and equation numbers.
Numerical method	Patankar, S.V. (1980). Numerical Heat Transfer and Fluid Flow.	General reference for FVM and linearized heat transfer practice, cited by the paper.

End of documentation.